

Sprint #1: Implementação dos comandos de entrada do SOP (Sistema de Orçamento Pessoal)

1. Objectivo (Sprint Goal)

Implementar os sete comandos de entrada do SOP forme especificação do projecto no enunciado.

2. Tarefas (Sprint Backlog)

Este Sprint tem um total de oito tarefas, nomeadamente:

1. Criar o projecto no Portugol Studio ou outras ferramentas que suportam a linguagem Portugol para Pseudocódigo.
2. Implementar o comando inserir movimento, ou um conjunto de movimentos.
3. Implementar o comando para mostrar todos os movimentos ocorridos numa determinada data.
4. Implementar o comando para remover um determinado movimento.
5. Implementar o comando para calcular o balanço numa determinada data, incluindo todos os movimento ocorridos nessa data.
6. Implementar o comando para alterar o valor mínimo do balanço que não invalida um orçamento, cujo valor por defeito é 0.
7. Implementar o comando para verificar a validade do orçamento.
8. Implementar o comando para determinar a primeira data em que uma determinada despesa se torna comportável, sem invalidar o orçamento.

Note que para este Sprint, o foco é apenas o reconhecimento de comandos inseridos pelo utilizador conforme especificado nos requisitos do projecto e a extracção dos argumentos para as variáveis de domínio adequado.

Para ilustrar a operacionalização da Sprint para as várias tarefas, vamos ilustrar as actividades para a tarefas 1 e 2, nas subsecções seguintes.

Tarefa #1: Criar o projecto no Portugol Studio

Comece por abrir a ferramenta Portugol Studio:

Computação Científica, ano lectivo 2025/2026

Aula Prática: Apoio a materialização do Sprint #1 do projecto.

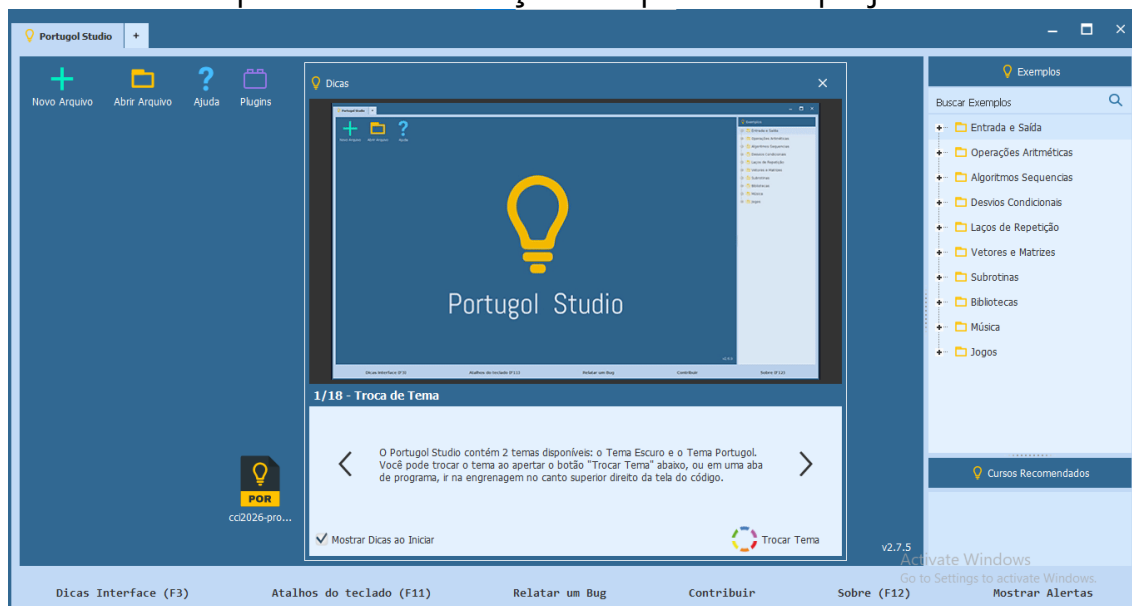


Figura 1: Tela inicial do Portugal Studio.

Fecha a tela de **Dicas** e clique no botão **Novo Arquivo** para abrir um novo ficheiro de código-fonte:

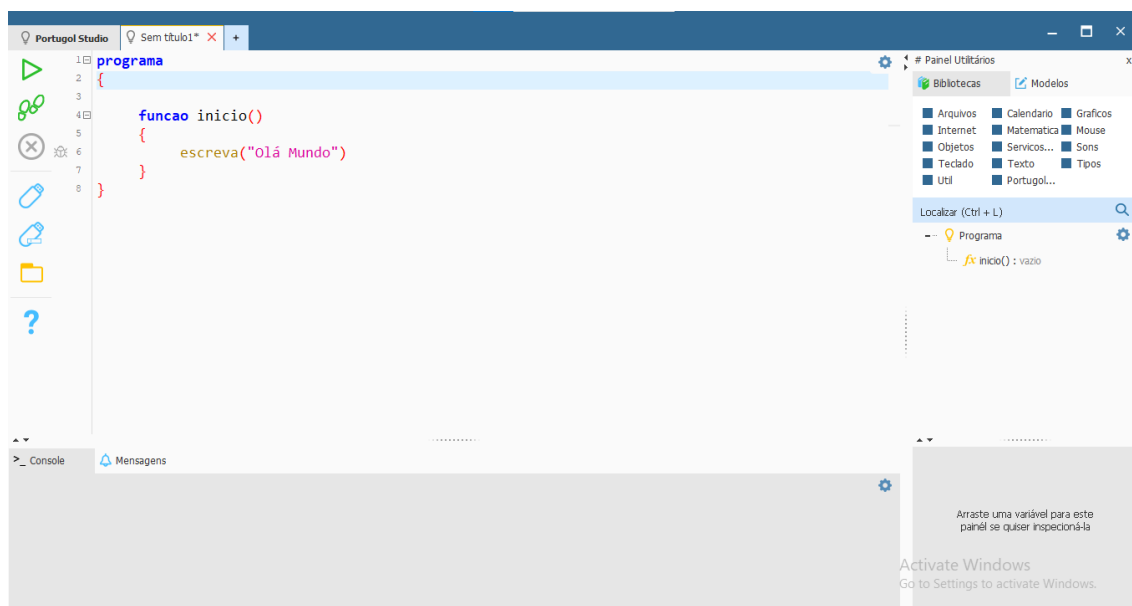


Figura 2: Novo ficheiro de código-fonte aberto.

Guarde o ficheiro de código-fonte, para tal clique no botão **Salvar** (), na janela **Save** escolha a área Desktop (Ambiente de Trabalho), crie nova pasta (clique no botão Criar Nova Pasta ou Create New Folder, nomeie e abra), dê o nome ao ficheiro de código-fonte de `cci2026-sop-gXX`, onde XX é o número do grupo definido na lista partilhada pelo(a) Delegado(a) da turma e na turma Google Classroom:

Computação Científica, ano lectivo 2025/2026

Aula Prática: Apoio a materialização do Sprint #1 do projecto.

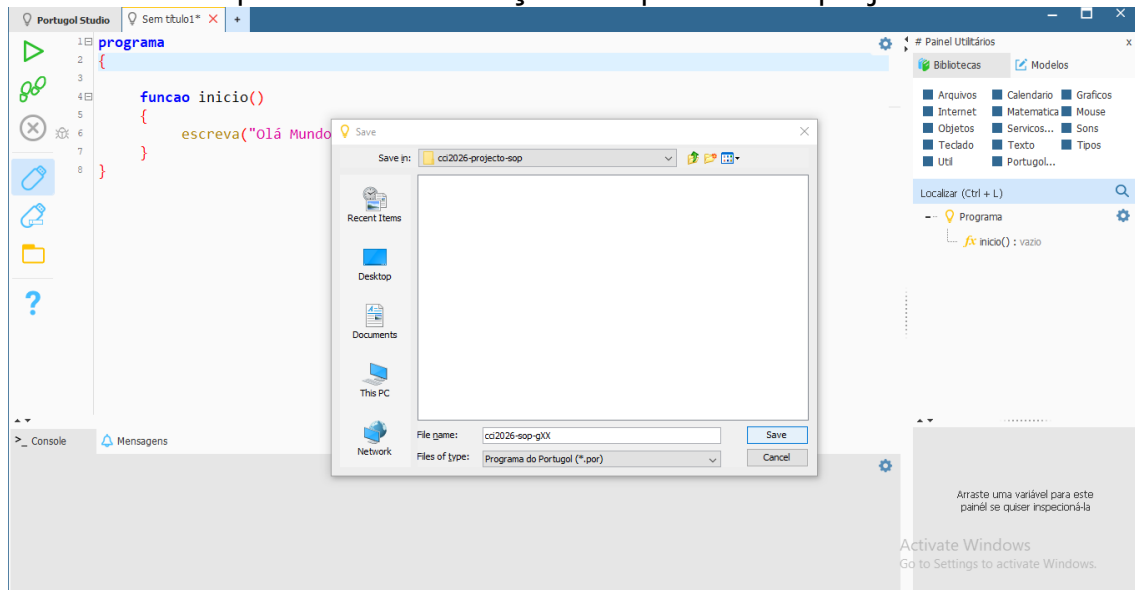


Figura 3: Tela de amostra usada para guardar o ficheiro de código-fonte.

Agora que o ficheiro de código-fonte já está guardado no seu computador, adicione comentário de documentação, para tal posiciona o cursor na primeira linha antes da palavra programa e pressione a tecla ENTER (esta acção fará a palavra programa passar para segunda linha):

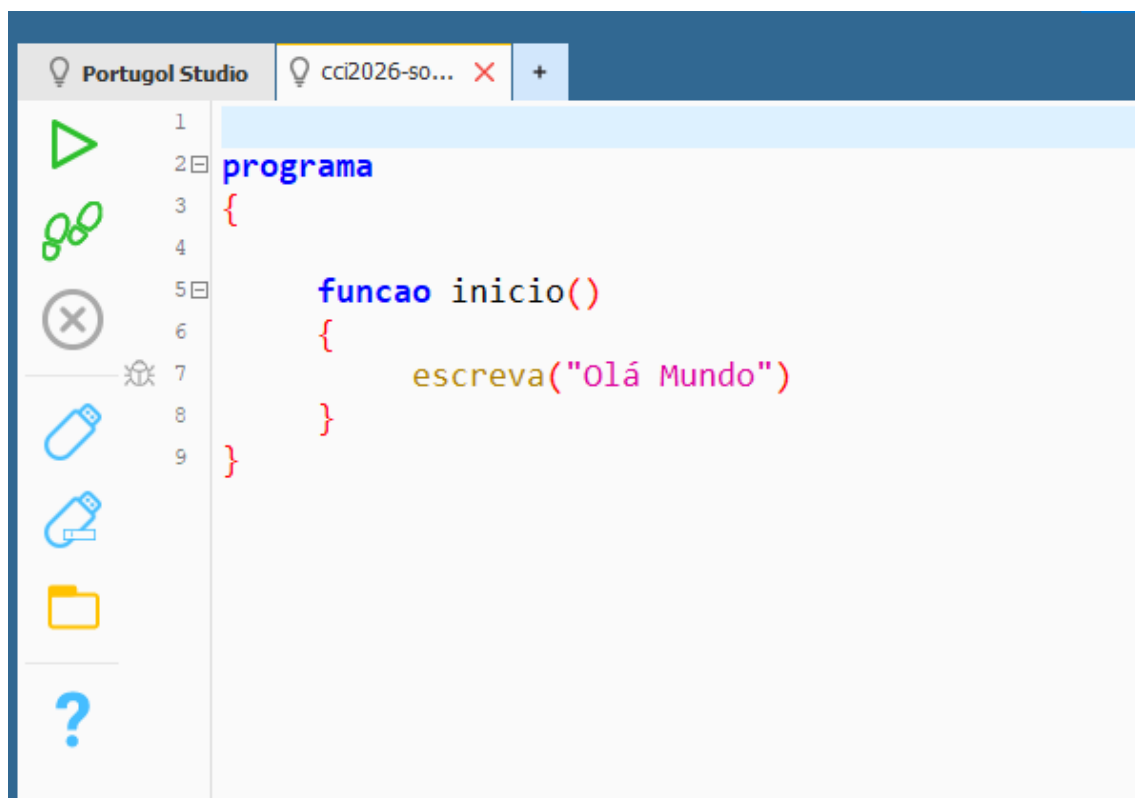


Figura 4: Cursor posicionado na primeira linha do ficheiro de código-fonte.

Agora posiciona o cursor na primeira linha, digite `/**` e pressione a tecla ENTER (essa acção vai gerar o comentário de documentação):

Computação Científica, ano lectivo 2025/2026

Aula Prática: Apoio a materialização do Sprint #1 do projecto.

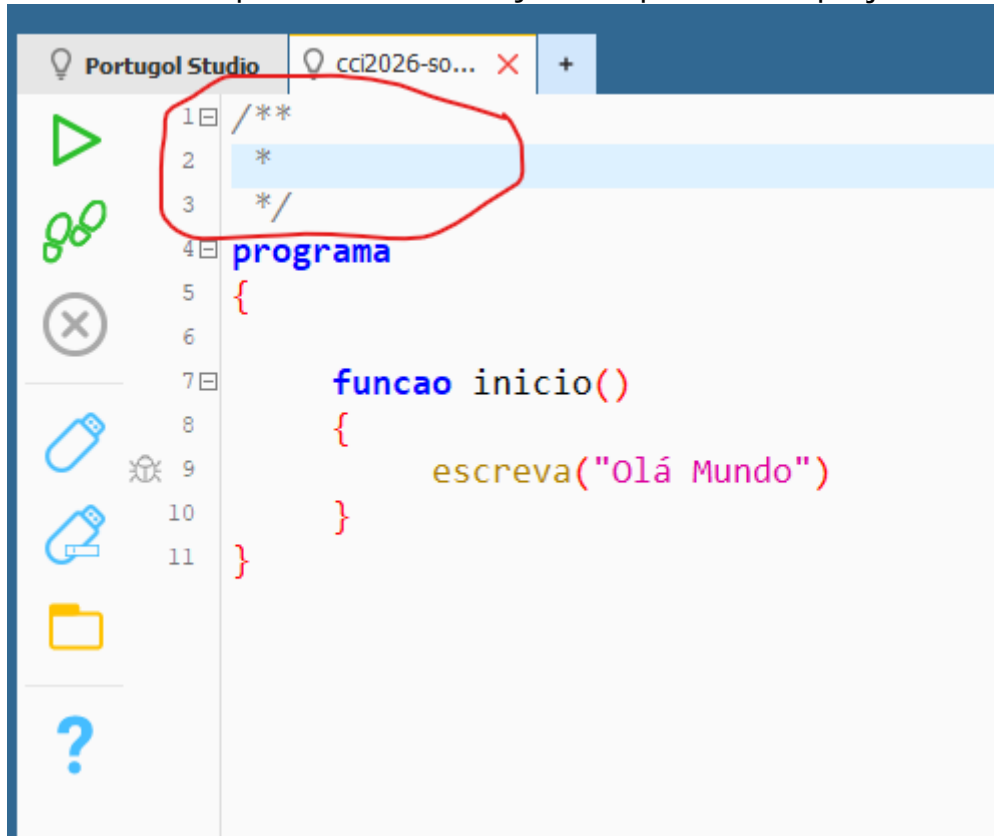


Figura 5: Comentário de documentação gerado (marcado com linha vermelha).

Identifique o ficheiro de código-fonte com os autores (grupo, a turma e número de matrícula e nome dos membros do grupo) e descrição do projecto:

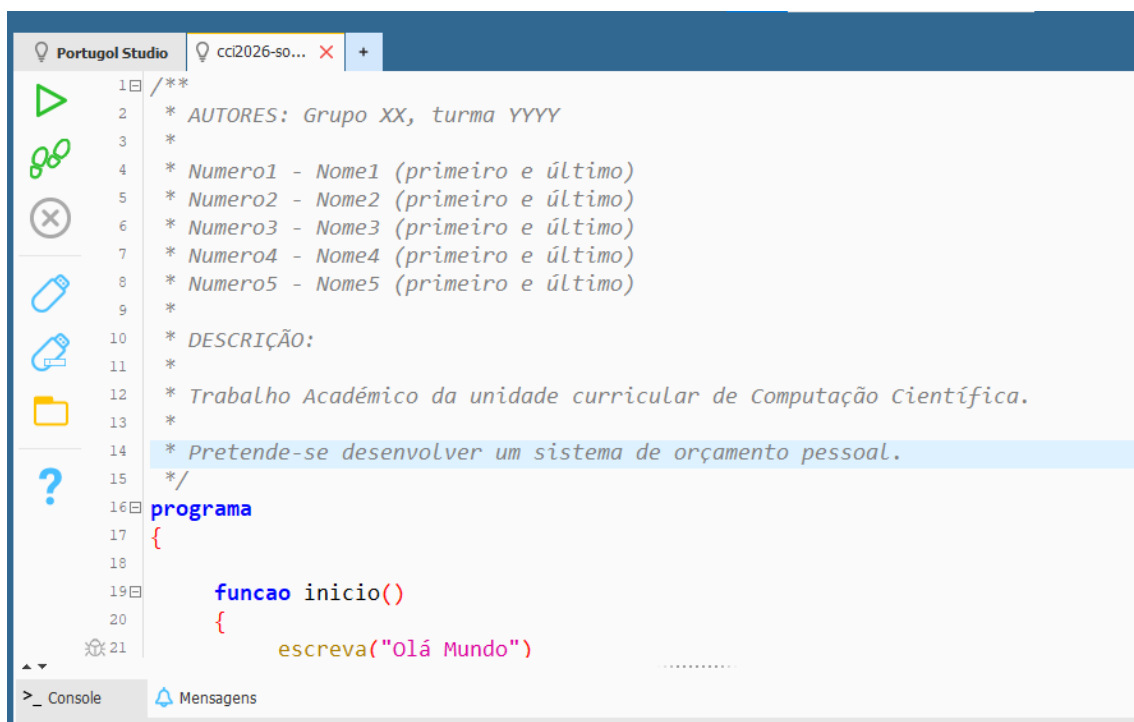


Figura 6: Modelo de comentário de documentação a ser usado.

Computação Científica, ano lectivo 2025/2026

Aula Prática: Apoio a materialização do Sprint #1 do projecto.

Aguarde as alterações ao clicar no botão **Save**.

Até aqui considere por concluída a primeira tarefa do Sprint #1.

As próximas tarefas são incrementadas a estas (metodologia incremental) e para cada tarefa/acção de testar e visualizar resultados.

Tarefa #2: Implementar o comando inserir movimento, ou um conjunto de movimentos

Comece por escrever o algoritmo em descrição narrativa para implementar essa funcionalidade (do scrum **Task Breakdown**):

Passo 1: receber um texto.

Passo 2: extrair no texto o primeiro caractere.

Passo 3: determinar se o caractere corresponde a um comando válido.

Passo 4: se for um comando válido, então

Passo 4.1: extrair os argumentos do comando.

Passo 4.2: apresentar os argumentos de forma individual.

Passo 5: senão, então

Passo 5.1: apresentar a mensagem comando inválido.

Passo 6: voltar ao passo 1.

Observações sobre o algoritmo anterior:

- Alguns passos podem ser detalhados, por exemplo o passo 2 (extrair é um subalgoritmo que tem como objectivo extrair uma subcadeia. Sugerimos o uso da função `extrair_subtexto` da biblioteca `Texto`):

Computação Científica, ano lectivo 2025/2026

Aula Prática: Apoio a materialização do Sprint #1 do projecto.



Figura 7: Descrição da função `extrair_subtexto` da biblioteca `Texto`.

Perceba que as bibliotecas disponíveis no Portugol Studio a documentação está em **Ajuda**. Por exemplo para a função `extrair_subtexto` da biblioteca `Texto`: a primeira linha está a declaração da função (para saber os domínios e contradomínio da função), a seguir vem a descrição com exemplo (para saber o que faz a função e perceber a sua utilidade) e a seguir a descrição dos parâmetros (domínios), do retorno (contradomínio) e os autores.

Idealmente, precisa ler a documentação e testar exemplos para dominar e poder aplicar como parte do seu algoritmo.

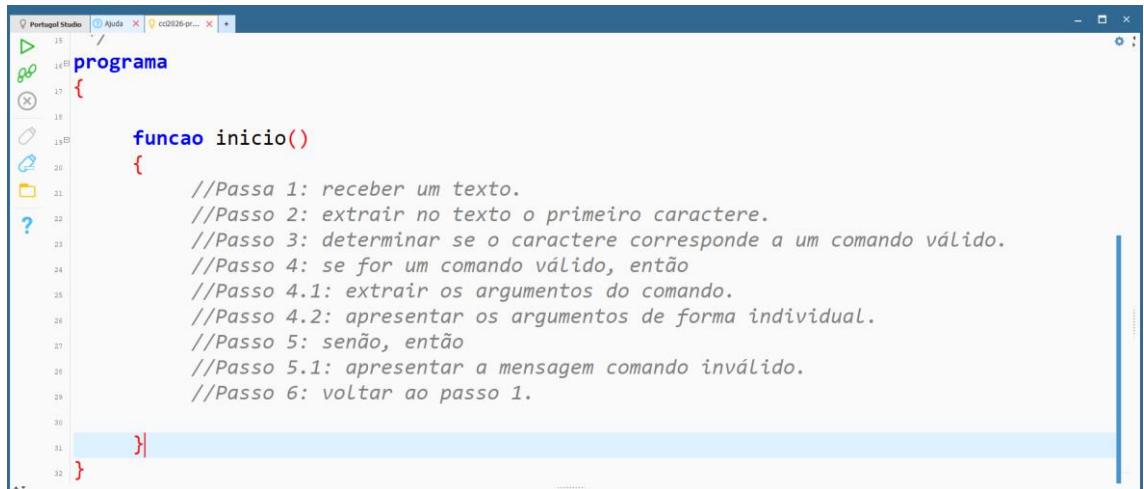
- Os comandos estão especificados no enunciado do trabalho, determinar se o caracter é um comando válido consiste em verificar se é “i”, “l”, “r”, “b”, “m”, “v” ou “w”.
- Extrair os argumentos do comandos consiste em: (i) colocar os valores, passados na mesma linha que o comando e separados por um espaço em branco, em uma variável independente do tipo cadeia, (ii) converter a cadeia para tipo de dado apropriado (por exemplo o primeiro argumento do comando “i” é o valor ou montante e esse deve ser um número ou inteiro ou real e não uma cadeia).
- Perceba que o passo 6 garante a repetição dos passos anteriores de forma infinita (o algoritmo termina apenas se interromper a execução). Assim sendo, esse passo é uma forma de descrever estrutura de repetição infinita (a condição é sempre verdadeiro).

Agora podemos passar para execução (lembrando que durante a execução a programação é em par, deve haver reunião diária de no máximo 15 minutos, relatório de actividade diário enviado no Grupo WhatsApp por cada membro, estimativa diária do tempo restante para conclusão do sprint feita pelo líder do grupo e criar o gráfico de tempo real de burn-down atualizada pelos membros do grupo):

Computação Científica, ano lectivo 2025/2026

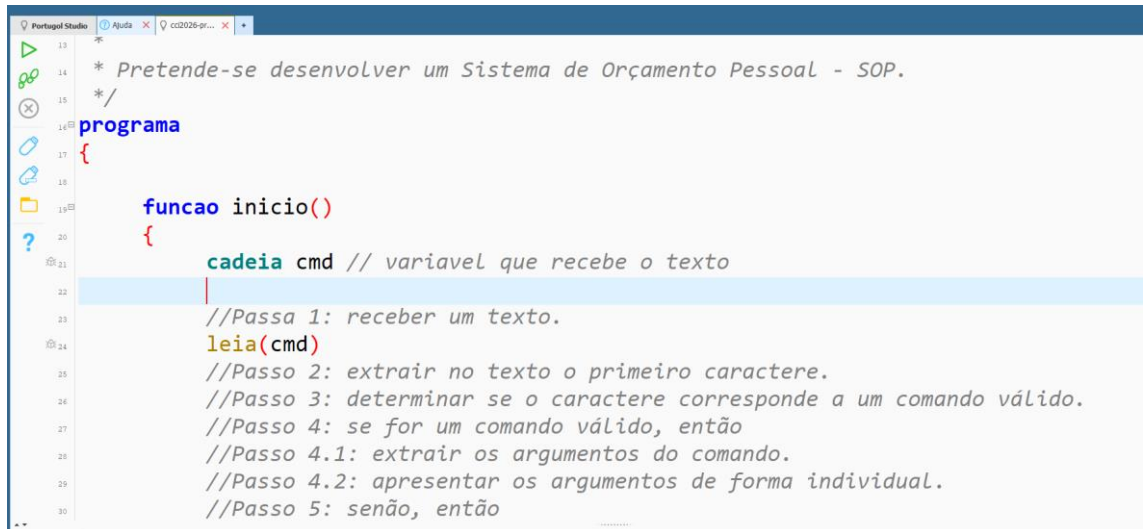
Aula Prática: Apoio a materialização do Sprint #1 do projecto.

1. Adicione o algoritmo acima em comentário no ficheiro de código-fonte no Portugol Studio:



```
13 programa
14 {
15     //Passo 1: receber um texto.
16     //Passo 2: extrair no texto o primeiro caractere.
17     //Passo 3: determinar se o caractere corresponde a um comando válido.
18     //Passo 4: se for um comando válido, então
19     //Passo 4.1: extrair os argumentos do comando.
20     //Passo 4.2: apresentar os argumentos de forma individual.
21     //Passo 5: senão, então
22     //Passo 5.1: apresentar a mensagem comando inválido.
23     //Passo 6: voltar ao passo 1.
24 }
25
```

2. Agora implemente cada passo do algoritmo, começando pelo passo 1:



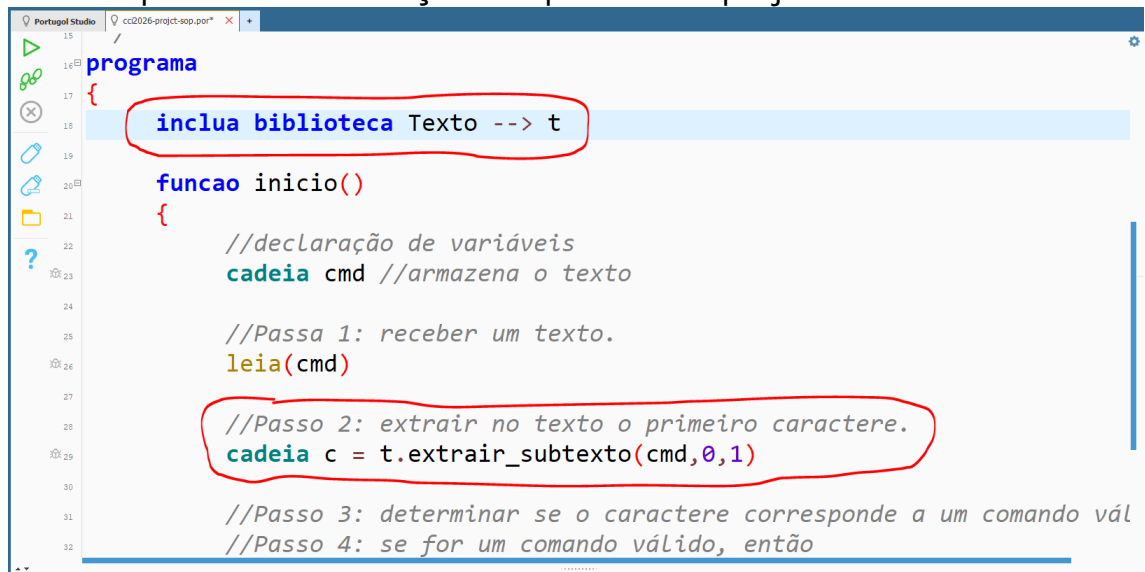
```
13 * Pretende-se desenvolver um Sistema de Orçamento Pessoal - SOP.
14 * /
15 programa
16 {
17     funcao inicio()
18     {
19         cadeia cmd // variavel que recebe o texto
20         //Passo 1: receber um texto.
21         leia(cmd)
22         //Passo 2: extrair no texto o primeiro caractere.
23         //Passo 3: determinar se o caractere corresponde a um comando válido.
24         //Passo 4: se for um comando válido, então
25         //Passo 4.1: extrair os argumentos do comando.
26         //Passo 4.2: apresentar os argumentos de forma individual.
27         //Passo 5: senão, então
28     }
29 }
30
```

O domínio de valor que podemos armazenar texto é a **cadeia** (tipo de dado), o comando usado para receber dado é o **leia**.

3. Implementação do passo 2:

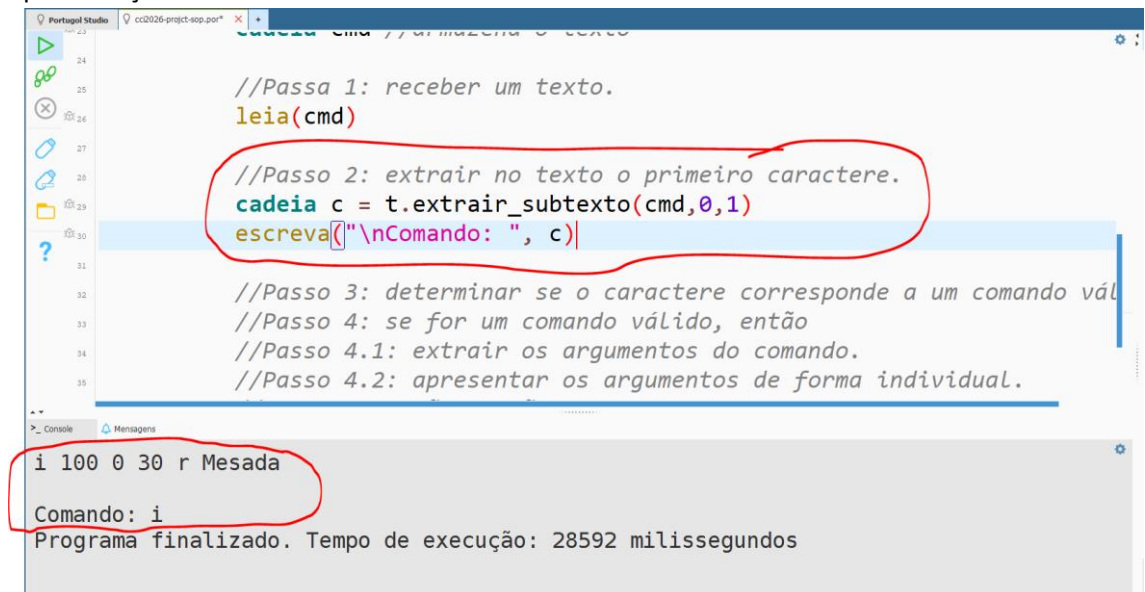
Computação Científica, ano lectivo 2025/2026

Aula Prática: Apoio a materialização do Sprint #1 do projecto.



```
15 programa
16 {
17     inclui biblioteca Texto --> t
18
19     funcao inicio()
20     {
21         //declaração de variáveis
22         cadeia cmd //armazena o texto
23
24         //Passo 1: receber um texto.
25         leia(cmd)
26
27         //Passo 2: extrair no texto o primeiro caractere.
28         cadeia c = t.extrair_subtexto(cmd,0,1)
29
30         //Passo 3: determinar se o caractere corresponde a um comando vál
31         //Passo 4: se for um comando válido, então
32     }
```

Repare que não implementamos o algoritmo para extracção de subtexto – usamos a função `extrair_subtexto` da biblioteca `Texto` usando o alias `t` (nome substituto). Essa função recebe a cadeia e o intervalo de caracteres correspondente o subtexto desejado (repare que o intervalo `[posicao-inicial, posicao-final[` em que o primeiro caractere corresponde a posição zero e o último ao tamanho da cadeia menos um,... dado o intervalo, o caractere corresponde a posição final não é incluído no subtexto... assim o intervalo `[0, 1[` extrair apenas o primeiro caractere). Testa a apresentação do conteúdo da variável `c`:



```
23 cadeia cmd //armazena o texto
24
25 //Passo 1: receber um texto.
26 leia(cmd)
27
28 //Passo 2: extrair no texto o primeiro caractere.
29 cadeia c = t.extrair_subtexto(cmd,0,1)
30 escreva("\nComando: ", c)
31
32 //Passo 3: determinar se o caractere corresponde a um comando vál
33 //Passo 4: se for um comando válido, então
34 //Passo 4.1: extrair os argumentos do comando.
35 //Passo 4.2: apresentar os argumentos de forma individual.
```

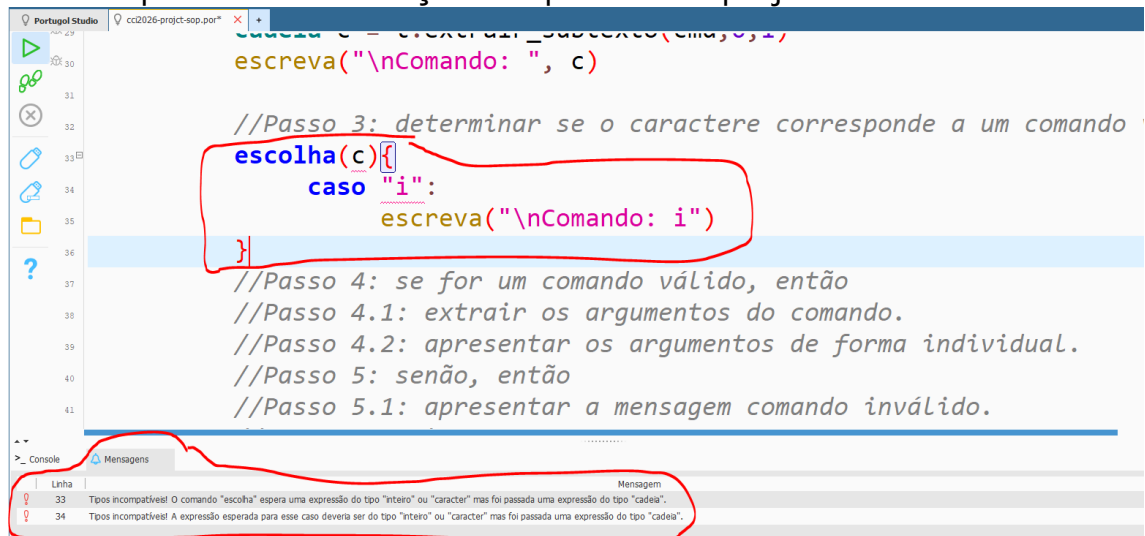
i 100 0 30 r Mesada
Comando: i
Programa finalizado. Tempo de execução: 28592 milissegundos

Repare que adicionamos uma linha para apresentar o conteúdo da variável `c`... executando, inserindo um comando (novo movimento de receita) e apresentado o comando digitado (apenas o primeiro caractere do texto lido). Assim sendo o passo 2 está implemento com sucesso... pode eliminar a instrução `escreva`, por ser desnecessário. Também poderíamos testar usando a ferramenta Debugging do Portugol Studio (assunto para discussão na aula de laboratório).

- Podemos agora implementar o passo 3:

Computação Científica, ano lectivo 2025/2026

Aula Prática: Apoio a materialização do Sprint #1 do projecto.



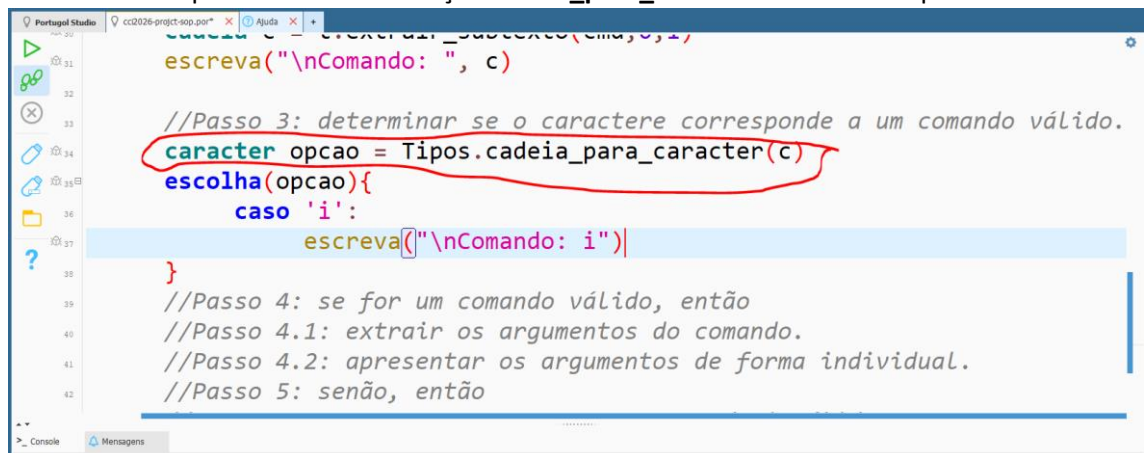
The screenshot shows a code editor with a switch statement. The switch statement is on line 34, with a case for 'i'. The console messages are on lines 33 and 34. The messages indicate a type mismatch between the variable 'c' and the expected types 'inteiro' or 'caracter'.

```
30 escreva("\nComando: ", c)
31
32 //Passo 3: determinar se o caractere corresponde a um comando
33 escolha(c){
34     caso "i":
35         escreva("\nComando: i")
36     }
37 //Passo 4: se for um comando válido, então
38 //Passo 4.1: extrair os argumentos do comando.
39 //Passo 4.2: apresentar os argumentos de forma individual.
40 //Passo 5: senão, então
41 //Passo 5.1: apresentar a mensagem comando inválido.
```

Console Messages:

- 33 Tipos incompatíveis! O comando "escolha" espera uma expressão do tipo "inteiro" ou "caracter" mas foi passada uma expressão do tipo "cadeia".
- 34 Tipos incompatíveis! A expressão esperada para esse caso deveria ser do tipo "inteiro" ou "caracter" mas foi passada uma expressão do tipo "cadeia".

Repare que o comando escolha aceita apenas tipo caractere e número inteiro e o a variável **c** é do tipo **cadeia**... ou seja, são incompatíveis. Assim sendo, precisamos converter o tipo cadeia em caractere e comparar com constante caractere... Para conversão de tipo utilizaremos a função **cadeia_para_caractere** biblioteca Tipos:



The screenshot shows the same code editor with the switch statement. The variable 'c' is now converted to 'opcao' using the 'cadeia_para_caractere' function. The console messages are no longer present.

```
30 escreva("\nComando: ", c)
31
32 //Passo 3: determinar se o caractere corresponde a um comando válido.
33 caracter opcao = Tipos.cadeia_para_caractere(c)
34 escolha(opcao){
35     caso 'i':
36         escreva("\nComando: i")
37     }
38 //Passo 4: se for um comando válido, então
39 //Passo 4.1: extrair os argumentos do comando.
40 //Passo 4.2: apresentar os argumentos de forma individual.
41 //Passo 5: senão, então
```

Repare que realizando a conversão o erro desaparece... agora adiciona os casos para os restantes comandos:

Computação Científica, ano lectivo 2025/2026

Aula Prática: Apoio a materialização do Sprint #1 do projecto.

```
Portugol Studio cc2026-project-sop.por x Ajuda x +
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52

caracter opcao = Tipos.cadeia_para_caracter(c)
escolha(opcao){
    caso 'i':
        escreva("\nComando: i") pare
    caso 'l':
        escreva("\nComando: l") pare
    caso 'r':
        escreva("\nComando: r") pare
    caso 'b':
        escreva("\nComando: b") pare
    caso 'm':
        escreva("\nComando: m") pare
    caso 'v':
        escreva("\nComando: v") pare
    caso 'w':
        escreva("\nComando: w") pare
    caso contrario:
        escreva("\ncomando inválido!")
}

//Passo 4: se for um comando válido, então
```

São, no total, sete comandos de entrada...

5. Repare que o Passo 5.1, já está implementado:

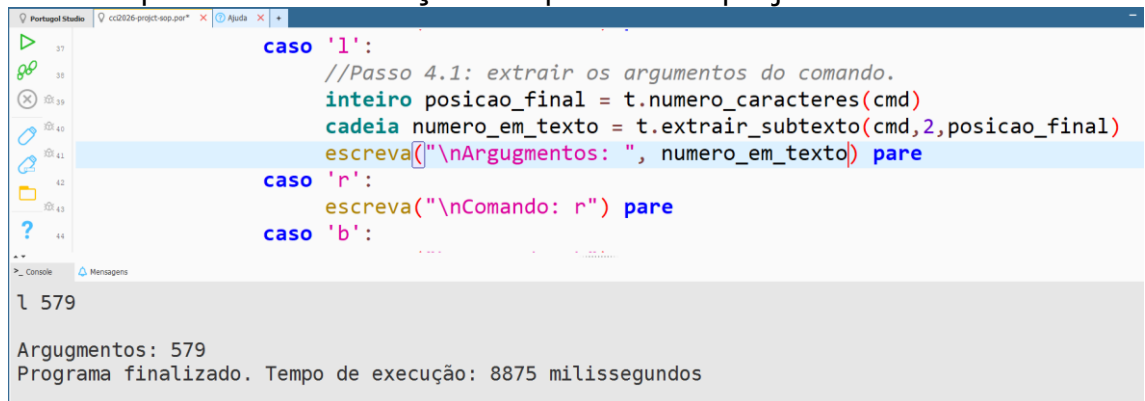
```
Portugol Studio cc2026-project-sop.por x Ajuda x +
39
40
41
42
43
44
45
46
47
48
49
50
51
52

caso 'r':
    escreva("\nComando: r") pare
caso 'b':
    escreva("\nComando: b") pare
caso 'm':
    escreva("\nComando: m") pare
caso 'v':
    escreva("\nComando: v") pare
caso 'w':
    escreva("\nComando: w") pare
//Passo 5.1: apresentar a mensagem comando inválido.
caso contrario:
    escreva("\nComando inválido!")
}
```

6. Agora vamos implementar os 4.1 e 4.2 para o comando "l", comando para mostrar todos os movimentos ocorridos numa determinada data tem o seguinte formato:

Computação Científica, ano lectivo 2025/2026

Aula Prática: Apoio a materialização do Sprint #1 do projecto.



```
Portugal Studio cc02026-project-esp-2025-2026 Ajuda x +
37 caso 'l':
38     //Passo 4.1: extrair os argumentos do comando.
39     inteiro posicao_final = t.numero_caracteres(cmd)
40     cadeia numero_em_texto = t.extrair_subtexto(cmd, 2, posicao_final)
41     escreva("\nArgumentos: ", numero_em_texto) pare
42
43 caso 'r':
44     escreva("\nComando: r") pare
45
46 caso 'b':
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

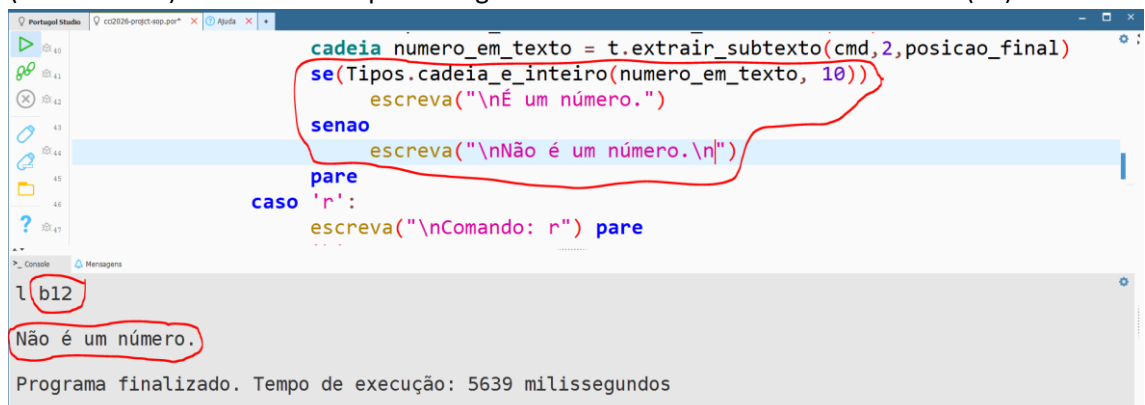
Console Mensagens

1 579

Argumentos: 579
Programa finalizado. Tempo de execução: 8875 milissegundos

No que no comando “l xx”, antes do argumento temos um espaço, portanto o primeiro caractere do argumento começa na posição 2 e o último caractere na posição tamanho menos um, assim precisamos saber o tamanho da cadeia (ou seja, quantos caracteres tem a cadeia), para isso usamos a função **numero_caracteres** da biblioteca **Texto**,... com esses dados é possível extrair a cadeia de caracteres correspondente ao número.

Agora que já obtemos a cadeia que pode ser um número, precisamos verificar se é de facto um número, para tal vamos usar a função **cadeia_e_inteiro** (cadeia cad, inteiro base) da biblioteca **Tipos**, onde base pode ser 2 (binária), 10 (decimal) ou 16 (hexadecimal). No nosso caso para o argumento desse comando é base decimal (10):



```
Portugal Studio cc02026-project-esp-2025-2026 Ajuda x +
40 cadeia numero_em_texto = t.extrair_subtexto(cmd, 2, posicao_final)
41 se(Tipos.cadeia_e_inteiro(numero_em_texto, 10))
42     escreva("\nÉ um número.")
43 senao
44     escreva("\nNão é um número.\n")
45 pare
46 caso 'r':
47     escreva("\nComando: r") pare
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

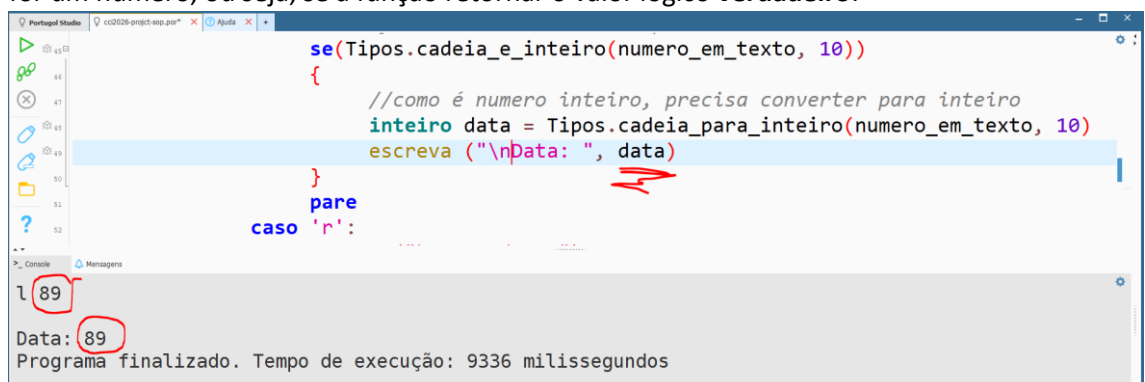
Console Mensagens

1 b12

Não é um número.

Programa finalizado. Tempo de execução: 5639 milissegundos

Note que a função retorna **verdadeiro** caso a cadeia seja um número e retorna **falso** caso contrário. Logo para condição de teste não preciso testar, porque a função já retorna um valor lógico. Só podemos implementar acções adicionais se o argumento for um número, ou seja, se a função retornar o valor lógico **verdadeiro**:



```
Portugal Studio cc02026-project-esp-2025-2026 Ajuda x +
43 se(Tipos.cadeia_e_inteiro(numero_em_texto, 10))
44 {
45     //como é numero inteiro, precisa converter para inteiro
46     inteiro data = Tipos.cadeia_para_inteiro(numero_em_texto, 10)
47     escreva("\nData: ", data)
48 }
49 pare
50 caso 'r':
51     escreva("\nComando: r") pare
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Console Mensagens

1 89

Data: 89

Programa finalizado. Tempo de execução: 9336 milissegundos

Assim concluímos os passos 4.1 e 4.2 para comando lista.

O grupo precisa dar continuidade ao trabalho para conclusão do Sprint #1 (TAREFAS PENDENTES):

- Extrair os argumentos dos restantes comandos
- Implementar o Passo 6.
- Refactorar – modularização por funções/procedimentos

As tarefas pendentes é para serem realizadas pelo grupo com o apoio do docente nas aulas do laboratório ou em horário agendados.

Bom trabalho!!!!